

Practical 5

Aim:

Perform Create, Read, Update, and Delete (CRUD) Operations Using MongoDB.

Problem Definition:

Scenario: A growing e-commerce/analytics platform needs to store and manage large volumes of unstructured and semi-structured data — such as student records, product catalogues, and event logs — that do not fit neatly into rigid relational schemas.

As a Data Analytics Engineer, your responsibility is to install and configure MongoDB, create a database and collection, perform CRUD (Create, Read, Update, Delete) operations on documents, apply query filters, and use the MongoDB aggregation pipeline to analyze the stored data, demonstrating how NoSQL document databases manage data at scale in Big Data environments.

Objectives:

To install and configure MongoDB, create a database and collection, insert documents (single and multiple), read/query documents using various filters, update existing documents, delete documents and collections, and use the MongoDB aggregation pipeline for data analysis.

Observation / Output (Screenshots or command outputs)

1. Install and configure MongoDB.

Install MongoDB Community Edition and start the MongoDB service, then connect using the mongosh shell.

```
PS D:\BDA\BDA-SHP-2026> mongosh
Current Mongosh Log ID: 6aab9cae61995d2f18b5f898
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&app
Name=mongosh+2.5.0
Using MongoDB:      8.0.8
Using Mongosh:      2.5.0
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-09-15T22:48:13.640+05:30: Access control is not enabled for the database. Read and write access to
  data and configuration is unrestricted
-----

test> 
```

2. Create a database and collection.

```
sS D:\BDA\BDA-SHP-2026>
Database: bda_practical5 | Collection: students

CREATE: insertOne
Command: db.students.deleteMany({})
Command: db.students.insertOne({"student_id":"24DIT001","name":"Aarav Shah","department":"Data Engineering",
,"year":2,"semester":4,"cgpa":8.7,"skills":["Python","MongoDB","Hadoop"],"city":"Ahmedabad"})
{
  acknowledged: true,
  insertedId: ObjectId('6aab9bfd1a99797cf4b5f899')
}
Output:
{
  student_id: '24DIT001',
  name: 'Aarav Shah',
  department: 'Data Engineering',
  year: 2,
  semester: 4,
  cgpa: 8.7,
  skills: [
    'Python',
    'MongoDB',
    'Hadoop'
  ],
  city: 'Ahmedabad'
}
```

3. Insert documents (single and multiple).

```
CREATE: insertMany
Command: db.students.insertMany([{"student_id":"24DIT002","name":"Diya Patel","department":"Data Engineering",
,"year":2,"semester":4,"cgpa":9.1,"skills":["Python","Spark","SQL"],"city":"Vadodara"}, {"student_id":"24DIT003",
"name":"Kabir Mehta","department":"Computer Engineering","year":2,"semester":4,"cgpa":7.9,"skills":["Java",
"MongoDB","Docker"],"city":"Surat"}, {"student_id":"24DIT004","name":"Myra Desai","department":"Computer Engineering",
"year":3,"semester":6,"cgpa":8.4,"skills":["Python","Spark","AWS"],"city":"Ahmedabad"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6aab9bfd1a99797cf4b5f89a'),
    '1': ObjectId('6aab9bfd1a99797cf4b5f89b'),
    '2': ObjectId('6aab9bfd1a99797cf4b5f89c')
  }
}
```

4. Read / Query documents using various filters.

READ: all students

Command: `db.students.find({}, { _id: 0 }).sort({ cgpa: -1 })`

Output:

```
{
  student_id: '24DIT002',
  name: 'Diya Patel',
  department: 'Data Engineering',
  year: 2,
  semester: 4,
  cgpa: 9.1,
  skills: [
    'Python',
    'Spark',
    'SQL'
  ],
  city: 'Vadodara'
}
{
  student_id: '24DIT001',
  name: 'Aarav Shah',
  department: 'Data Engineering',
  year: 2,
  semester: 4,
  cgpa: 8.7,
  skills: [
    'Python',
    'MongoDB',
    'Hadoop'
  ],
  city: 'Ahmedabad'
}
```

READ: CGPA >= 8.5

Command: `db.students.find({ cgpa: { $gte: 8.5 } }, { _id: 0, student_id: 1, name: 1, cgpa: 1 })`

Output:

```
{
  student_id: '24DIT001',
  name: 'Aarav Shah',
  cgpa: 8.7
}
{
  student_id: '24DIT002',
  name: 'Diya Patel',
  cgpa: 9.1
}
```

5. Update existing documents.

```
UPDATE: one document
Command: db.students.updateOne({ student_id: "24DIT001" }, { $set: { cgpa: 8.9 }, $addToSet: { skills: "Py
Mongo" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Output:
{
  student_id: '24DIT001',
  name: 'Aarav Shah',
  department: 'Data Engineering',
  year: 2,
  semester: 4,
  cgpa: 8.9,
  skills: [
    'Python',
    'MongoDB',
    'Hadoop',
    'PyMongo'
  ],
  city: 'Ahmedabad'
}
```

```
UPDATE: many documents
Command: db.students.updateMany({ department: "Computer Engineering" }, { $set: { active: true } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

6. Delete documents and collections.

```
DELETE: one document
Command: db.students.deleteOne({ student_id: "24DIT004" })
{
  acknowledged: true,
  deletedCount: 1
}
```

7. Use MongoDB aggregation pipeline for data analysis.

```

AGGREGATION: department summary
Command: db.students.aggregate([ { $group: { _id: "$department", student_count: { $sum: 1 }, average_cgpa: { $avg: "$cgpa" }, highest_cgpa: { $max: "$cgpa" } } }, { $sort: { average_cgpa: -1 } }, { $project: { _id: 0, department: "$_id", student_count: 1, average_cgpa: { $round: ["$average_cgpa", 2] }, highest_cgpa: 1 } } ])
Output:
{
  student_count: 2,
  highest_cgpa: 9.1,
  department: 'Data Engineering',
  average_cgpa: 9
}
{
  student_count: 2,
  highest_cgpa: 8.4,
  department: 'Computer Engineering',
  average_cgpa: 8.15
}

```

Result:

MongoDB was successfully installed and configured, and a database with a collection was created to store student records. Documents were inserted using both insertOne and insertMany, and were retrieved using queries with filters, projections, sorting, and limits. Existing documents were updated using updateOne and updateMany, and unwanted documents and the collection were removed using deleteOne, deleteMany, and drop. The aggregation pipeline was used to group student records by branch and compute the average CGPA and student count per branch, demonstrating how MongoDB manages and analyzes semi-structured data at scale.

GitHub: [BDA-SHP-2026/Practical5 at main · AubaidAhmedSaiyed/BDA-SHP-2026](https://github.com/AubaidAhmedSaiyed/BDA-SHP-2026)

Key Questions / Analysis:

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like documents (internally in a binary format called BSON) instead of rows and columns in fixed tables. Unlike relational databases, which enforce a rigid, predefined schema and split related data across multiple normalized tables joined by foreign keys, MongoDB allows each document within a collection to have a different structure, and related data can be embedded directly within a single document. This schema flexibility makes MongoDB well suited to unstructured or evolving data, while relational databases remain stronger for use cases that need strict schema enforcement and complex multi-table transactions. MongoDB also scales horizontally by sharding data across multiple servers, whereas traditional relational databases typically scale vertically by adding more resources to a single server.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update, and Delete — the four basic operations that can be performed on data stored in any database. Create adds new documents (insertOne, insertMany), Read retrieves and filters existing documents (find, findOne), Update modifies fields in existing documents (updateOne, updateMany), and Delete removes documents or entire collections (deleteOne, deleteMany, drop). These four operations are fundamental because together they cover the complete life cycle of any piece of data in a system — from creation, through being read and modified, to eventual removal — and virtually every application feature, from a user signing up to an admin editing a record, ultimately maps to one of these operations.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores documents internally as BSON (Binary JSON), a binary-encoded serialization of JSON-like documents. BSON extends plain JSON by adding extra data types that JSON does not natively support, such as Date, ObjectId, Int32/Int64, Double, Decimal128, and Binary data, and it stores length prefixes and type information alongside each field. This makes BSON documents faster to parse and traverse than text-based JSON, since the database does not need to scan character by character to determine field boundaries or types. Every document also gets a unique `_id` field (an ObjectId by default) that acts as its primary key within the collection. Because BSON is still document-shaped rather than table-shaped, related or nested data (like an array of skills or an embedded address object) can be stored directly within a single document.

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline is MongoDB's framework for processing and transforming documents through a sequence of stages, where the output of one stage becomes the input to the next — similar to a Unix pipeline. Common stages include `$match` (filter documents, similar to a WHERE clause), `$group` (group documents by a field and compute aggregates such as `$sum`, `$avg`, `$min`, `$max`), `$sort` (order the results), `$project` (reshape documents by including, excluding, or computing fields), `$limit`/`$skip` (paginate results), and `$lookup` (perform a join-like operation with another collection). Because each stage operates on the output of the previous one, complex multi-step analytical queries — such as computing the average CGPA per branch, as shown in Task 7 — can be expressed as a single readable pipeline rather than multiple separate queries processed in application code.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases like MongoDB are preferable when the data is unstructured, semi-structured, or has a schema that changes frequently and cannot be easily fixed in advance, when the application needs to scale horizontally across many commodity servers to handle very large volumes of reads/writes, and when the access pattern favors retrieving one related, self-contained document (rather than joining many normalized tables) for low-latency reads. They are also a good fit for rapid application development, where the data model is expected to evolve, and for use cases such as content management, product catalogues, real-time analytics, and IoT/event logging. Relational (SQL) databases remain the better choice when the application requires strong ACID transactional guarantees across multiple related tables, complex multi-table joins, and strict schema/data-integrity enforcement — such as in banking or financial systems.

Supplementary Problems:

1. Create an index on a collection and compare query performance. Before creating the index, MongoDB performs a COLLSCAN (a full collection scan), examining every document to find matches. After creating the index with `createIndex({ branch: 1 })`, the same query uses an IXSCAN (index scan), which examines far fewer documents (`totalDocsExamined` drops close to `totalKeysExamined`) and returns results faster — the improvement becomes especially significant as the collection grows larger.

Output:

```

INDEX: query statistics before and after city index
Command: db.students.dropIndex("city_1")
Command: db.students.find({ city: "Ahmedabad" }).explain("executionStats")
Command: db.students.createIndex({ city: 1 })
Command: db.students.find({ city: "Ahmedabad" }).explain("executionStats")
Output:
{
  index: 'city_1',
  documentsExaminedBefore: 4,
  documentsExaminedAfter: 2,
  executionTimeMillisBefore: 0,
  executionTimeMillisAfter: 9
}

```

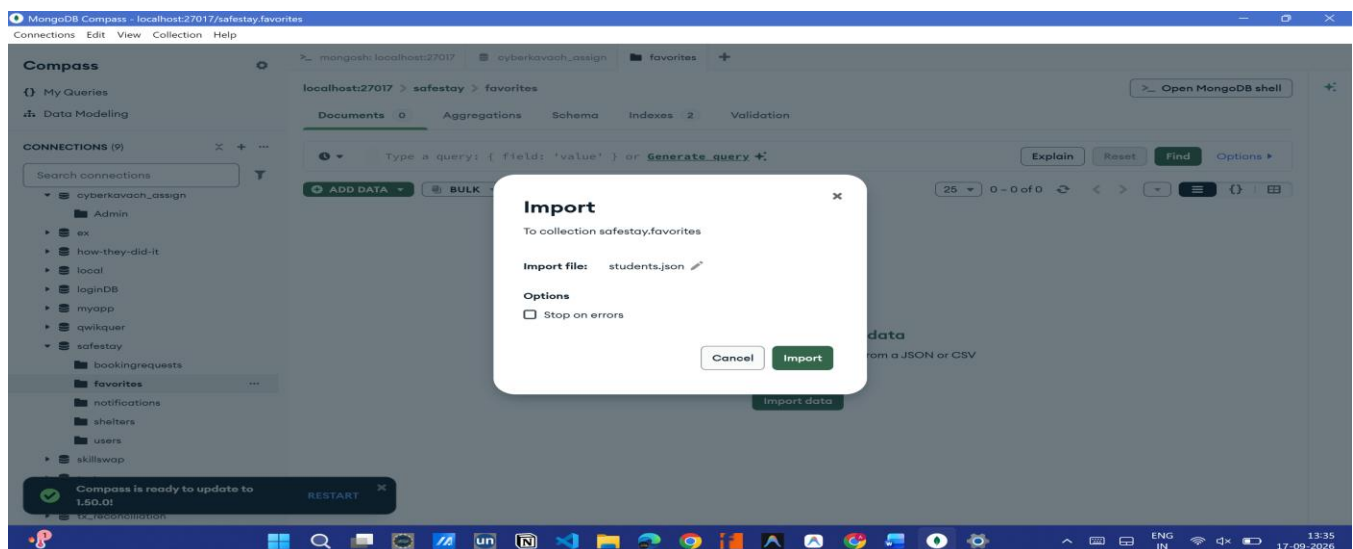
2. Use the aggregation pipeline to generate a summary report. *Output:*

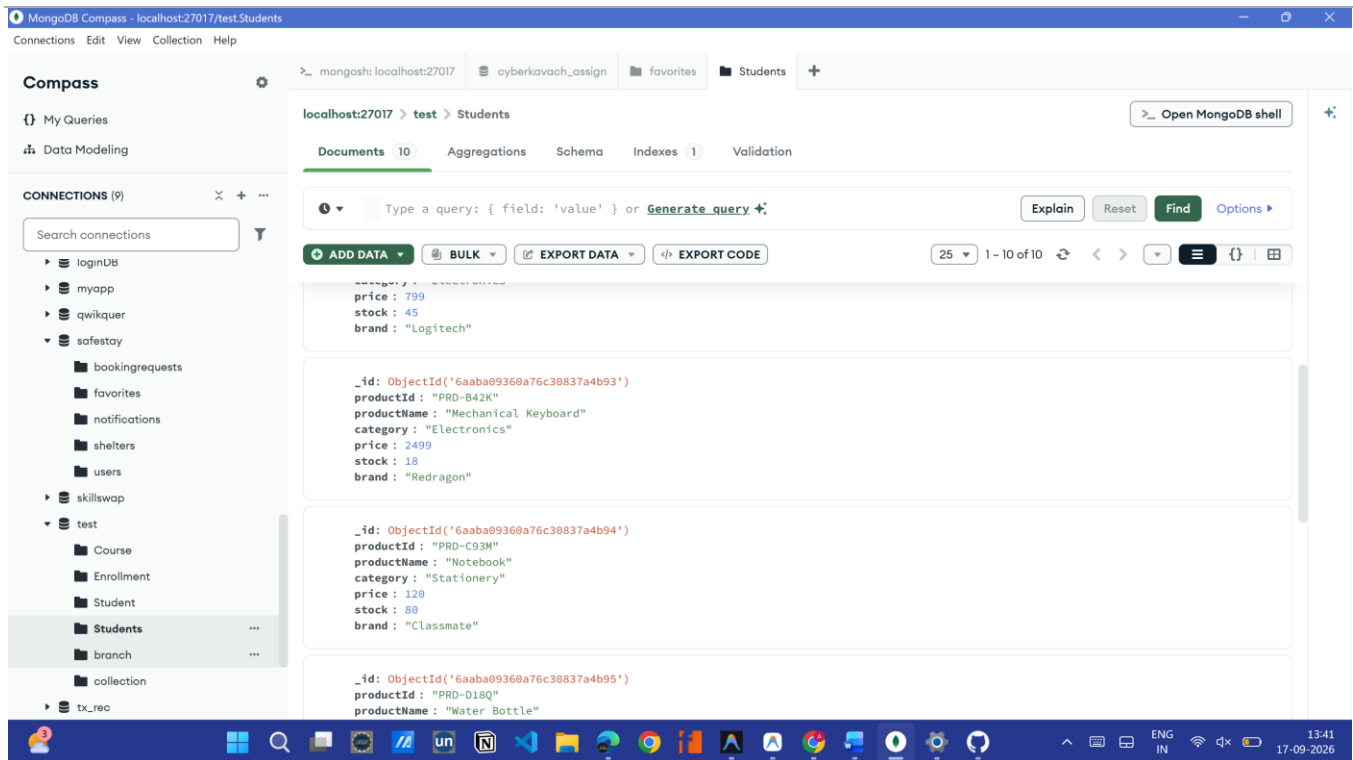
```

AGGREGATION: department summary
Command: db.students.aggregate([{$group: { _id: "$department", student_count: { $sum: 1 }, average_cgpa: { $avg: "$cgpa" }, highest_cgpa: { $max: "$cgpa" } } }, {$sort: { average_cgpa: -1 } }, {$project: { _id: 0, department: "$_id", student_count: 1, average_cgpa: { $round: ["$average_cgpa", 2] }, highest_cgpa: 1 } }])
Output:
{
  student_count: 2,
  highest_cgpa: 9.1,
  department: 'Data Engineering',
  average_cgpa: 9
}
{
  student_count: 2,
  highest_cgpa: 8.4,
  department: 'Computer Engineering',
  average_cgpa: 8.15
}

```

3. Import a JSON dataset into MongoDB and perform analytical queries.





Tools / Technology used:

MongoDB 6.x / 7.x, MongoDB Shell (mongosh), MongoDB Compass (GUI), VS Code, Linux/Ubuntu Environment.

Conclusion:

Through this practical, MongoDB was installed and configured, and a database and collection were created to manage semi-structured student data. CRUD operations were performed end-to-end — documents were inserted singly and in bulk, queried using filters, projections, sorting, and limits, modified using `updateOne` and `updateMany`, and removed using `deleteOne`, `deleteMany`, and `drop`. The aggregation pipeline was then used to group and summarize the data, computing statistics such as average CGPA per branch. This demonstrated how MongoDB, as a NoSQL document database, provides the schema flexibility, horizontal scalability, and analytical querying capability needed to manage unstructured and semi-structured data efficiently in Big Data environments, compared to the rigid schema constraints of traditional relational databases.